

Lab 5 : Cryptographie

I. Task1 : Hash function

```
(kali@vbox)-[~/Bureau]
$ nano Plain.txt

(kali@vbox)-[~/Bureau]
$ cat Plain.txt
security lab

(kali@vbox)-[~/Bureau]
$ man openssl

(kali@vbox)-[~/Bureau]
$ openssl dgst -sha1 Plain.txt
SHA1(Plain.txt)= 8d678009606a414a6d163a1d96d832fddde1d3e4

(kali@vbox)-[~/Bureau]
$ nano Plain.txt

(kali@vbox)-[~/Bureau]
$ cat Plain.txt
Security lab

(kali@vbox)-[~/Bureau]
$ openssl dgst -sha1 Plain.txt
SHA1(Plain.txt)= 87a7acddb0d51bf22ed471c32215b1990f95aacb
```

- On crée et modifie le fichier Plain.txt avec la commande `nano Plain.txt`. Ensuite on affiche avec `cat Plain.txt` pour vérifier que le fichier a bien été enregistré avec notre valeur.
- On utilise ensuite la commande `openssl dgst -sha1 Plain.txt` pour générer le hash de la valeur du fichier Plain.txt avec l'algorithme SHA1.
- On ouvre de nouveau l'éditeur de texte pour modifier le fichier Plain.txt. Puis on refait de nouveau un chiffrement de Plain.txt avec le SHA1.
- Enfin on constate que les deux hash des deux versions différentes de Plain.txt sont bien différents car ils n'ont pas les mêmes valeurs. Cela est parce que les deux fichiers d'origines n'étaient pas les mêmes, ainsi à l'issue de l'algorithme de hashage le SHA1 ne peut être le même.

II. Task2 : Symmetric encryption

a)

Maxime Sourdin
Angel Velasco
ING4

```
(kali@vbox)-[~/Bureau]
$ openssl enc -des-cbc -in Plain.txt -out Cipher.txt -base64 -k password123
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

```
(kali@vbox)-[~/Bureau]
$ cat Cipher.txt
U2FsdGVkX1+1oqR166jXmitghtmDIQs5sP4QuGemFcw=
```

Pour chiffrer le fichier *Plain.txt* avec l'algorithme DES-CBC on utilise la commande suivante : ``openssl enc -des-cbc -in Plain.txt -out Cipher.txt -base64 -k password123``. Les options `-in` et `-out` permettent de donner le fichier à chiffrer et dans quel fichier stocker la sortie. L'option `-base64` permet de stocker le hash en base 64 dans le fichier *Cipher.txt* et l'option `-k` permet de spécifier le mot de passe utilisé pour le chiffrement.

b)

```
(kali@vbox)-[~/Bureau]
$ openssl enc -des-cbc -d -in Cipher.txt -out NewPlain.txt -base64 -k password123
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

```
(kali@vbox)-[~/Bureau]
$ cat NewPlain.txt
Security lab

(kali@vbox)-[~/Bureau]
$ diff Plain.txt NewPlain.txt
```

Pour déchiffrer le fichier *Cipher.txt*, nous utilisons la commande suivante :

```
openssl enc -des-cbc -d -in Cipher.txt -out NewPlain.txt -base64 -k password123
```

Dans cette commande, l'option `-d` indique que nous souhaitons effectuer une opération de déchiffrement.

Le fichier *Cipher.txt* est spécifié comme entrée grâce à l'option `-in`, tandis que le résultat du déchiffrement est enregistré dans le fichier *NewPlain.txt* à l'aide de l'option `-out`.

L'option `-base64` est utilisée car le fichier chiffré est encodé en Base64.

En affichant le contenu du fichier *NewPlain.txt* à l'aide de la commande `cat`, on constate que le message obtenu est identique à celui du fichier *Plain.txt*.

Cela est normal car DES est un algorithme de chiffrement symétrique, ce qui signifie que la même clé secrète est utilisée pour le chiffrement et le déchiffrement.

Maxime Sourdin
Angel Velasco
ING4

III. Task3 : Asymmetric encryption

Dans cette partie l'objectif est d'encrypter un message avec une clé publique. On se repose sur l'utilisation d'une paire de clés : une publique pour chiffrer le message et la clé privée pour déchiffrer le message. Soit seul le détenteur de la clé privée est capable de lire le contenu original.

a. Generate RSA keys with length of 2048 bits; privMyName.key is a name of the private key

Premièrement on génère une clé privée RSA d'une longueur de 2048 bits, qui est une taille standard de sécurité.

```
vboxuser@DebianVM:~$ openssl genrsa -out privMyName.key 2048
vboxuser@DebianVM:~$ ls
Cipher.txt  Downloads  Plain.txt  Videos
Cortex      elastic   privMyName.key  wazuh-agent_4.14.1-1_amd64.deb
Cypher.txt  Music     Public
Desktop     n8n      Shuffle
Documents   Pictures  Templates
vboxuser@DebianVM:~$ head privMyName.key
-----BEGIN PRIVATE KEY-----
MIIEugIBADANBgkqhkiG9w0BAQEFAASCBAQwggSgAgEAAoIBAQC4EhWLQ3k19NV
0Y8Qho7KN/A4osaCKounLW3zfJ4/WdXEKM/HMxVVr0Bc2uf06nu5v96JWBHVsvFV
AMHSNv7VEkAqh6FhqiM9mYVaj8ItSB4RhIuYVvPtQum0EtoN4TeIcVxYnQVwEnw
IphdUL06UN84mKrtppAitexUcaEQzDTreTaFsiNMwoY4aW7M/IJy8FTACSWHhi+3
wWJFxmHRKSSNG+duG/uLrmNCJVt8d5oyuNFaIhkRpi6d0ZAZU0blg366BefY25Zp
C3WPamk0E1Rrj4vLtpvIY70vbTFlik/6k4snzHYW6phvcKvnYBb1kVwXxmYINx6Z
YgH1ntvXAgMBAAECgf8PaSgWdJN7nNvDhm3IyCT/ycqUEkV6M+1A4PMAwWgKSx3
ocHgX0LYdQzEj+Ry9dLY540htCc0d9GDexCVMhwu8jSyMlgCRBlGfjsS/GyJ+jXi
PV6kbm+L8LHEEsu7fgPC+rvlgLuDhOQid3J0No1f+/8fXK5U62kALtTH6PtDQbfK
vboxuser@DebianVM:~$ S
```

b. Extract public key from privMyName.key; pubMyName.key is a name of the public key.

Une fois la clé privée générée, nous devons en extraire la clé publique. C'est cette clé qui pourra être partagée librement pour permettre à d'autres de nous envoyer des messages chiffrés.

```
~$ openssl rsa -in privMyName.key -pubout -out pubMyName.key
```

c. Encrypt the file Plain.txt with the public key, save the file encrypted as CipherRSA.txt.

Maxime Sourdin

Angel Velasco

ING4

Nous procédons maintenant au chiffrement d'un fichier texte nommé Plain.txt en utilisant la clé publique générée précédemment. Le résultat est stocké dans un fichier binaire , CipherRSA.txt.

```
vboxuser@DebianVM:~$ openssl pkeyutl -encrypt -inkey pubMyName.key -pubin -in Plain.txt -out CipherRSA.txt
vboxuser@DebianVM:~$ head CipherRSA.txt
a0`M0η$"x|P>0g)00(0)00"0Ur00J00E_0g[Df4t00*w0v000040$g)JWP0k000Y0},00q00x'000jL0{1000jBr"v0[0U0t000d00v00!
```

d. Decrypt the file CipherRSA.txt and save the new file as New2Plain.txt, compare the new file with Plain.txt, use : diff Plain.txt NewPlain.txt -q

Pour lire le message, nous utilisons la clé privée. Enfin, nous comparons le fichier déchiffré avec le fichier original pour vérifier qu'aucune donnée n'a été altérée.

```
vboxuser@DebianVM:~$ openssl pkeyutl -decrypt -inkey privMyName.key -in CipherRSA.txt -out New2Plaint.txt
vboxuser@DebianVM:~$ diff Plain.txt New2Plaint.txt -q
vboxuser@DebianVM:~$ diff Plain.txt New2Plaint.txt
```

La commande diff ne renvoie aucun résultat , les deux fichiers sont identiques.

IV. Task4 : Asymmetric/Symmetric Encryption, Digital Signature

1. Analyse du Scénario

L'objectif de cette tâche est de simuler un échange sécurisé entre deux entités, Alice et Bob, en combinant les avantages du chiffrement symétrique et asymétrique.

Chiffrement du message : Alice utilise une clé secrète symétrique k pour chiffrer le texte en clair M en un cryptogramme C.

Protection de la clé : La clé k est elle-même chiffrée avec la clé publique de Bob KpubB pour garantir que lui seul puisse la récupérer.

Authentification (Signature) : Alice signe la clé symétrique à l'aide de sa clé privée KprivA, permettant à Bob de vérifier l'origine du message grâce à la clé publique d'Alice KpubA.

Nous avons commencé par initialiser les clés nécessaires sur deux machines virtuelles distinctes

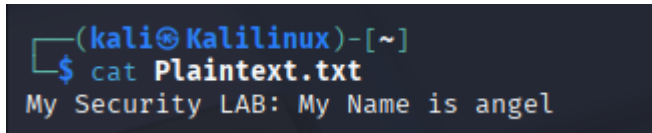
Clé symétrique : Génération d'une clé aléatoire de 128 bits (16 octets) destinée au chiffrement AES.

```
vboxuser@DebianVM:~$ openssl rand 16 > aes128.key
vboxuser@DebianVM:~$ head aes128.key
00] f00"F0Hg' :vboxuser@DebianVM:~$
```

a. Create Plaintext.txt

Nous avons créé un fichier nommé Plaintext.txt

Maxime Sourdin
Angel Velasco
ING4

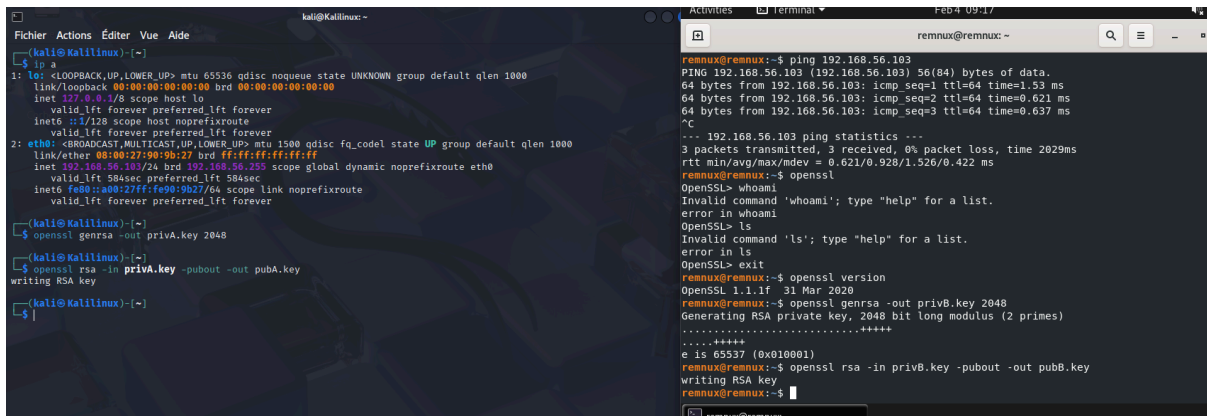


b. Generate RSA keys with length of 2048 bits: privA.key, pubA.key, privB.key and pubB.key (you can use keys

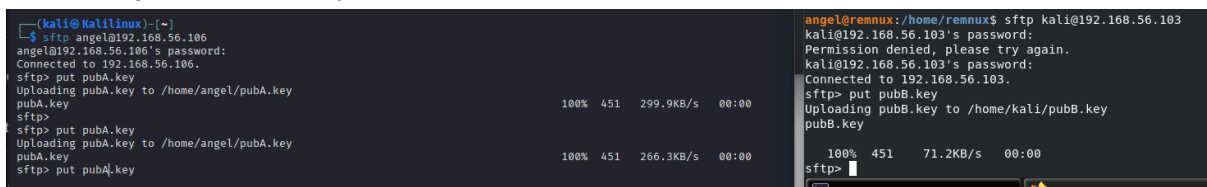
generated in Task 3 but you have to modify the file names).

Nous avons généré deux paires de clés asymétriques. Ainsi on peut établir une sécurité entre la machine Kali (A) et Remnux (B).

- Machine A : Génération de privA.key et extraction de pubA.key.
- Machine B : Génération de privB.key et extraction de pubB.key.



c. Exchange the public keys (use sftp or USB Flash)



On a utilisé le protocole SFTP pour l'échange. Pour permettre un chiffrement dans les 2 directions , la machine Kali a envoyé sa clé publique (pubA.key) vers le répertoire /home/angel/ de la machine cible, et la machine Remnux a transmis la sienne (pubB.key). Cet échange permet à chaque partie de posséder la clé publique de l'autre.

d. Encrypt PlaintextM.txt using symmetric algorithm

Pour protéger le contenu du message, nous utilisons un chiffrement symétrique.

Une clé de session est définie dans le fichier sym_key.txt.

Maxime Sourdin

Angel Velasco

ING4

Le fichier Plaintext.txt est chiffré pour produire Ciphertext.txt en utilisant l'option -kfile pour appeler le mot de passe et -base64.

```
(kali@Kalilinux)-[~]  
$ openssl enc -aes-128-cbc -base64 -in Plaintext.txt -out Ciphertext.txt -kfile sym_key.txt
```

e. Encrypt the symmetric key using asymmetric algorithm

La clé symétrique ne peut pas être envoyée en clair. Nous utilisons donc l'algorithme asymétrique pour chiffrer sym_key.txt avec la clé publique du destinataire (pubB.key). Le résultat est le fichier secret.key. Seule la clé privée du destinataire pourra l'ouvrir.

```
(kali@Kalilinux)-[~]  
$ openssl pkeyutl -encrypt -inkey pubB.key -pubin -in sym_key.txt -out secret.key
```

f. Generate the hash value of symmetric key using SHA1

(sym.sha1)

Afin de permettre au destinataire de vérifier que la clé symétrique n'a pas été modifiée, nous générons un hash de sym_key.txt en utilisant l'algorithme SHA1. Le résultat est stocké dans sym.sha1.

```
(kali@Kalilinux)-[~]  
$ openssl pkeyutl -sign -inkey privA.key -in sym.sha1 -out sym.sig
```

g. Sign sym.sha1, generate a file sym.sig

```
(kali@Kalilinux)-[~]  
$ openssl pkeyutl -verify -inkey pubA.key -pubin -sigfile sym.sig -in sym.sha1  
Signature Verified Successfully
```

Comme illustré dessus, nous faisons la signature du hash. En utilisant notre clé privée (privA.key), on signe sym.sha1 pour produire sym.sig. Cette signature lie l'identité de l'expéditeur au message envoyé.

h. Send the necessary files to your colleague that allows

decrypting your message, and verify your signature (

La validation vue au -dessus garantit que le message n'a pas été modifié et qu'il provient bien de l'expéditeur authentifié, permettant au destinataire de procéder au déchiffrement final.